

SMART RETAIL AI

Intelligent Retail Management & Forecasting Platform

Multi-Agent AI (LangGraph)	RAG + Pinecone	Azure OpenAI
Prophet Forecasting	FastAPI + React 19	Docker Deployment

Program	Left Shift Program 2026 – Data & AI (T5)
Domain	Intelligent Smart Retail System
Platform	Smart Retail AI
Report	Capstone Project Technical Report

. Table of Contents

1	Executive Summary
2	Project Objectives
3	System Architecture
4	Technology Stack
5	Multi-Agent AI System
6	Retrieval-Augmented Generation (RAG)
7	Machine Learning & Forecasting
8	REST API Design
9	Authentication & Security
10	Azure Cloud Integration
11	Deployment & Docker
12	Frontend Portals & Screenshots
13	Data Model & Business Workflows
14	Challenges & Optimizations
15	Future Enhancements
16	Conclusion

1. Executive Summary

Smart Retail AI is an end-to-end intelligent retail management platform that connects customers, retailers, warehouse administrators, and AI-powered business intelligence into one full-stack system. Instead of limiting retail automation to a simple storefront, the platform models a real retail supply chain: customers browse and order products, retailers manage inventory and restock requests, administrators control warehouse stock and approvals, and AI agents assist each role with contextual answers, analytics, forecasting, and anomaly detection.

The system combines a FastAPI backend, SQLAlchemy data models, JWT-secured authentication, React.js role-based portals, LangGraph multi-agent orchestration, Pinecone-backed Retrieval-Augmented Generation, Azure OpenAI language models, Prophet forecasting models, anomaly detection tools, Azure Blob Storage for sales backup access, and Docker-based deployment.

Capability	Description
Role-Based Portals	Separate protected portals for customers, retailers, and administrators.
AI Chatbots	Context-aware AI assistants powered by LangGraph and Azure OpenAI.
Demand Forecasting	Prophet-based 7-day and 30-day product demand forecasts.
Anomaly Detection	ML-backed identification of abnormal sales activity.
Cloud-Ready	Docker multi-stage build deployable on Azure App Service or Container Apps.
Secure Auth	JWT bearer tokens, bcrypt password hashing, and role-aware routing.

2. Project Objectives

Objective	Description
Role-Based Retail Operations	Provide separate protected portals for customers, retailers, and administrators with workflows tailored to each role.
AI-Assisted Customer Support	Answer customer questions about products, cart status, orders, payments, shipping, return policies, and company information.
Retailer Business Intelligence	Help retailers understand sales, stock, revenue, pending restocks, payments, and top-performing products using an analytics agent.
Demand Forecasting	Forecast product demand and sales trends using trained forecasting models for selected retail products.
Anomaly Detection	Identify abnormal sales activity using model-backed anomaly tools exposed through the retail ML agent.
Cloud Deployment	Package the React frontend and FastAPI backend into a Docker image suitable for Azure App Service or container deployment.
Secure Authentication	Protect APIs with JWT bearer tokens, hashed passwords, and role-aware frontend routing.

3. System Architecture

Smart Retail AI follows a layered architecture designed around real retail roles and AI-assisted decision-making. Each layer has a distinct responsibility, allowing independent scaling and maintenance.

Layer	Description
Frontend Layer	React 19 application with customer, retail, and admin portals, lazy-loaded routes, Tailwind styling, dashboards, catalog, cart, checkout, orders, and chatbots.
Backend Layer	FastAPI REST application exposing authentication, product, order, retail, warehouse, admin, dashboard, forecast, sales backup, and AI routes.
Authentication Layer	JWT bearer token flow with password hashing and protected API dependencies.
Multi-Agent Layer	LangGraph orchestrator routes user and retailer questions to QA, analytics, or retail ML agents.
RAG Layer	Pinecone vector search over SmartRetailAI policy and company documents, with local text-file fallback.
ML Layer	Prophet-based product forecasting and anomaly model loading through LangChain tools.
Database Layer	SQLAlchemy ORM with development SQLite defaults and production-ready database URL configuration.
Cloud Storage Layer	Azure Blob Storage SAS URL generation for sales backup CSV/text file downloads.
Deployment Layer	Multi-stage Docker build combining React static assets with the FastAPI runtime.

Workflow Overview

Step	Description
Step 1	User logs in as customer, retailer, or admin and receives a JWT access token.
Step 2	React ProtectedRoute allows access only to the role's portal.
Step 3	Frontend calls FastAPI endpoints for catalog, cart, orders, dashboard, restock, forecast, or chatbot features.
Step 4	FastAPI validates request data, resolves the current user, and opens a SQLAlchemy database session.
Step 5	AI requests enter /ai/chat for customers or /ai/retail-chat for retailers.
Step 6	LangGraph routes to QA, analytics, or retail ML nodes based on role and query keywords.
Step 7	Agents call RAG retrievers, database tools, forecast models, or anomaly tools.
Step 8	Results return to the frontend as dashboards, chat replies, reports, forecasts, or operational status updates.

4. Technology Stack

Category	Technology	Purpose
Backend Framework	FastAPI, Uvicorn	REST API development and async-ready Python service hosting.
Frontend	React 19, React Router, Tailwind CSS, Framer Motion	Role-based interactive portals and dashboard views.
Database	SQLAlchemy, SQLite / Azure SQL, pyodbc	ORM persistence with local development and Azure SQL compatibility.
Authentication	python-jose JWT, passlib bcrypt	Token authentication and secure password hashing.
LLM	AzureChatOpenAI	AI response generation for QA, analytics, and retail ML agents.
Orchestration	LangGraph StateGraph, ToolNode	Routing and tool-using agent workflows.
RAG / Vector DB	Pinecone, LangChain Retriever	Semantic retrieval over SmartRetail policy and support documents.
Embeddings	OpenAI / Azure OpenAI Embeddings	Vector representation for RAG documents.
Forecasting	Prophet, joblib, pandas	Product-level demand and sales forecast generation.
ML / Anomaly	scikit-learn, XGBoost, NumPy	Model-backed retail prediction and anomaly detection support.
Cloud Storage	Azure Blob Storage	Sales backup file access through time-limited SAS URLs.
Deployment	Docker multi-stage build	Build React frontend and serve it through the FastAPI container.

5. Multi-Agent AI System

The platform implements a LangGraph-based AI layer that separates customer support, retailer analytics, and machine-learning workflows. The orchestrator directs each question to the agent best suited to answer it, preventing a single chatbot from handling every concern blindly.

Agent	Role	Capability
Orchestrator Agent	Router	Inspects user role and query keywords, then routes to QA, analytics, or retail ML.
QA Agent	Customer Support	Uses RAG and database tools to answer policy, cart, order, payment, and product questions.
Analytics Agent	Retail Business Analyst	Uses database-backed tools for products, total sales, orders, low stock, top products, pending restocks, and payment summaries.
Retail ML Agent	Forecasting & Anomaly Specialist	Uses LangChain tools for product forecasts and anomaly checks over trained model artifacts.

Orchestration Logic

The orchestrator receives **query**, **role**, **user_id**, and **database session** state. Customer users are routed to the QA agent. Retail users are routed by keyword groups: policy and support questions go to QA, demand and anomaly questions go to the Retail ML agent, and sales or inventory questions go to the Analytics agent. The graph uses conditional edges from the router node and finishes after the selected specialist node returns a structured response.

6. Retrieval-Augmented Generation (RAG)

The RAG subsystem gives Smart Retail AI a stable knowledge base for company and policy questions. It stores and retrieves text documents covering company information, electronics policy, fashion guidance, home and kitchen essentials, health and personal care, loyalty, refunds, and shipping.

Document Ingestion Pipeline

Source	Text documents loaded from backend/rag/documents directory.
Chunking	RecursiveCharacterTextSplitter creates 500-character chunks with overlap.
Embeddings	OpenAI or Azure OpenAI embeddings convert chunks into vectors.
Storage	PineconeVectorStore uploads embedded chunks to the configured Pinecone index.

Retrieval Flow

Routing	QA agent identifies policy and company-information questions through keyword routing.
Query	retrieve_company_context() queries Pinecone with k=4 relevant chunks.
Fallback	If Pinecone is unavailable, the system falls back to token-overlap search across local .txt documents.
Injection	Retrieved context is injected into the system prompt so the LLM answers from known company information.

7. Machine Learning & Forecasting

Smart Retail AI includes two complementary ML paths: API-level demand prediction and agent-accessible retail forecasting tools. Together they provide both programmatic forecast endpoints and natural-language forecasting through the Retail ML agent.

Component	Details
Forecast API	GET /ai/forecast returns 7-day and 30-day forecast totals, trend direction, category, and current stock for selected products.
Forecast Models	Prophet model artifacts loaded with joblib from ml/ml_models using product IDs such as CE100, FA202, HK400, and HP303.
Retail ML Agent Tool	tool_forecast_product maps friendly product names to product IDs and returns predicted sales with a restocking recommendation.
Anomaly Tool	tool_anomaly_check loads anomaly models and evaluates abnormal sales activity using engineered features: quantity_sold, revenue, rolling_mean_7, lag_1, lag_7.
DemandForecastService	Loads demand_forecast_model.pkl and predicts demand from product, category, price, stock, discount, region, month, and day-of-week features.
Stock Interpretation	Prediction outputs are converted into business-friendly labels: Restocking Required, Low Stock, Stock Healthy.

8. REST API Design

The backend exposes a broad REST surface across core retail operations and AI capabilities. All endpoints are JWT-protected and validated through Pydantic schemas.

Method	Endpoint	Purpose
POST	/auth/register	Register a user with role and hashed credentials.
POST	/auth/login	Authenticate and issue a JWT access token.
GET	/auth/admins	List administrator users.
GET/POST/PATCH/DELETE	/products/warehouse	Warehouse product inventory management.
GET/POST/PATCH/DELETE	/products/retailer	Retailer product catalog management.
POST	/products/approve	Approve warehouse products into retailer inventory.
GET/POST/PATCH/DELETE	/orders/cart	Customer cart management.
POST	/orders/checkout	Create order and payment records from cart.
GET	/orders/my-orders	Customer order history.
GET	/orders/retailer-orders	Retailer order management.
PATCH	/orders/{order_id}/status	Update order status during fulfillment.
POST	/ai/chat	Customer chatbot using QA agent.
POST	/ai/retail-chat	Retailer chatbot using orchestrator + agents.
GET	/ai/forecast	Demand forecast output for featured products.
GET	/retail/restock	Retailer and admin restock request workflows.
GET	/backup/sales-csv	Generate Azure Blob SAS URL for sales backup.

9. Authentication & Security

Security Pattern	Implementation
JWT Authentication	create_access_token signs user payloads with HS256 and expiry from ACCESS_TOKEN_EXPIRE_MINUTES.
Password Security	passlib bcrypt hashes passwords and verifies login credentials without storing plaintext.
Protected Dependencies	Routes use get_current_user and get_db dependencies to enforce identity and provide scoped database access.
Role-Based UI	React ProtectedRoute limits customer, retail, and admin pages to allowedRoles.
Request Validation	Pydantic schemas validate API payloads; FastAPI returns validation errors for malformed requests.
Secret Management	Database URL, JWT secret, Azure OpenAI, Pinecone, and Azure Blob settings loaded from environment variables.
SAS URLs	Azure Blob sales backup access is granted through time-limited SAS tokens rather than public containers.
CORS Control	CORS middleware allows local frontend origins during development while keeping API behaviour explicit.

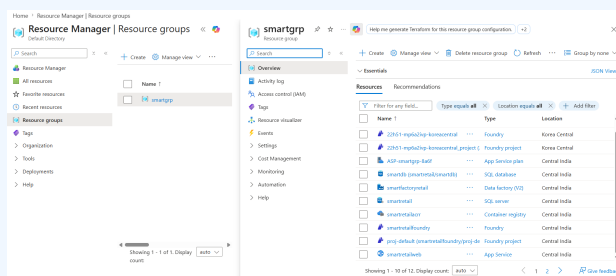
10. Azure Cloud Integration

The platform is built for Azure-native deployment, integrating multiple Azure services across AI, storage, database, and hosting layers.

Azure Service	How Used
Azure OpenAI	Powers QA, analytics, and retail ML agents; Azure/OpenAI embeddings support RAG depending on configuration.
Azure Blob Storage	Stores the sales backup blob and provides secure download links through generate_blob_sas.
Azure SQL	The backend uses DATABASE_URL and pyodbc support so the SQLAlchemy layer can move beyond local SQLite.
Azure App Service / Container	Docker image exposes FastAPI on the platform-provided PORT and serves the built React frontend.
Azure Data Factory (ADF)	Used for data pipeline orchestration and automated data movement across the retail data platform.
Azure ML / Model Registry	Hosts trained Prophet forecasting and anomaly detection models for production inference.

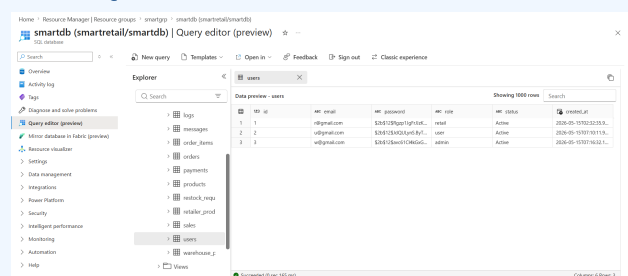
Azure Infrastructure Screenshots

Azure Resource Group



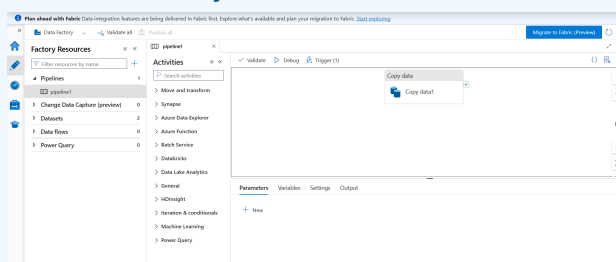
All Smart Retail AI services grouped under a single Azure resource group for unified management.

Azure SQL Database



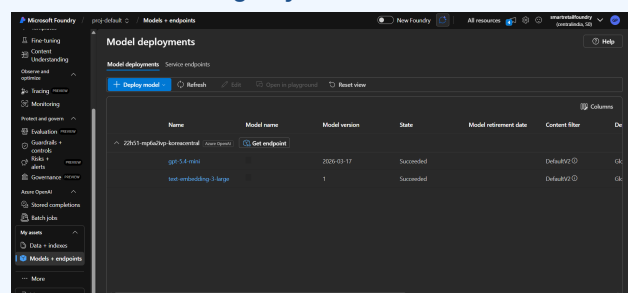
Azure SQL Database instance hosting the production retail data with pyodbc connectivity.

Azure Data Factory



ADF pipelines orchestrating data movement and ETL workflows for the retail analytics platform.

Azure ML Model Registry



Trained Prophet forecasting and anomaly detection models registered and versioned in Azure ML.

11. Deployment & Docker

The project uses a production-style multi-stage Dockerfile that bundles the React frontend and FastAPI backend into a single deployable container image.

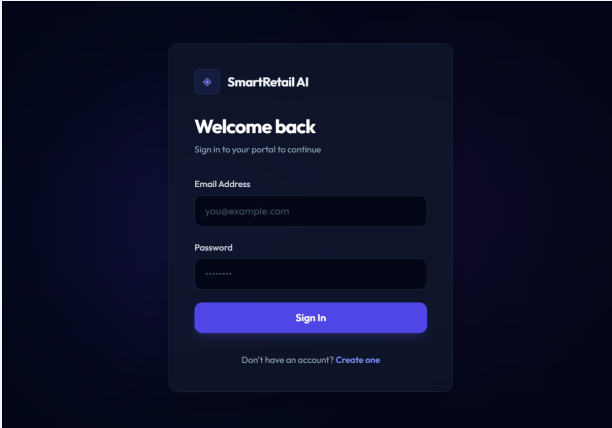
Stage / Component	Value	Purpose
Stage 1 – Build	node:20-bookworm-slim	Install frontend dependencies and build the React application.
Stage 2 – Runtime	python:3.11-slim-bookworm	FastAPI runtime with all Python dependencies from backend/requirements.txt.
System Packages	build tools, curl, unixODBC, ODBC Driver 18	Required for SQL Server connectivity and native extensions.
Static Assets	/app/frontend/build	Built React output copied into the container and served by FastAPI.
Entry Point	uvicorn app.main:app	Starts on 0.0.0.0 using the PORT environment variable.
Single Artifact	One container = API + Frontend	Simplifies capstone demonstration and Azure deployment.

12. Frontend Portals & Screenshots

The React 19 frontend is divided into three role-specific portals, each protected by ProtectedRoute components. Below are annotated screenshots covering every major screen of the application.

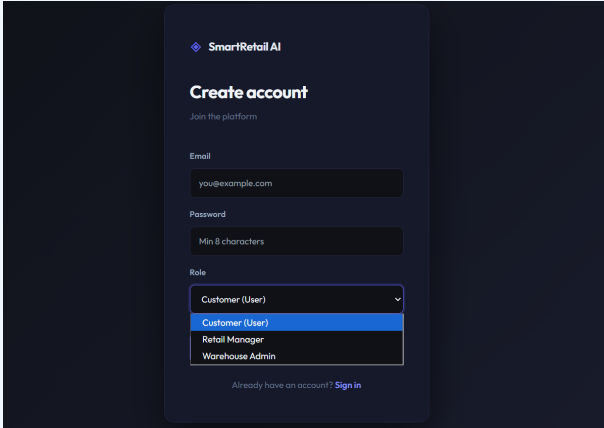
Authentication

Login Screen



JWT-secured login for all user roles. Role detection determines which portal the user is redirected to.

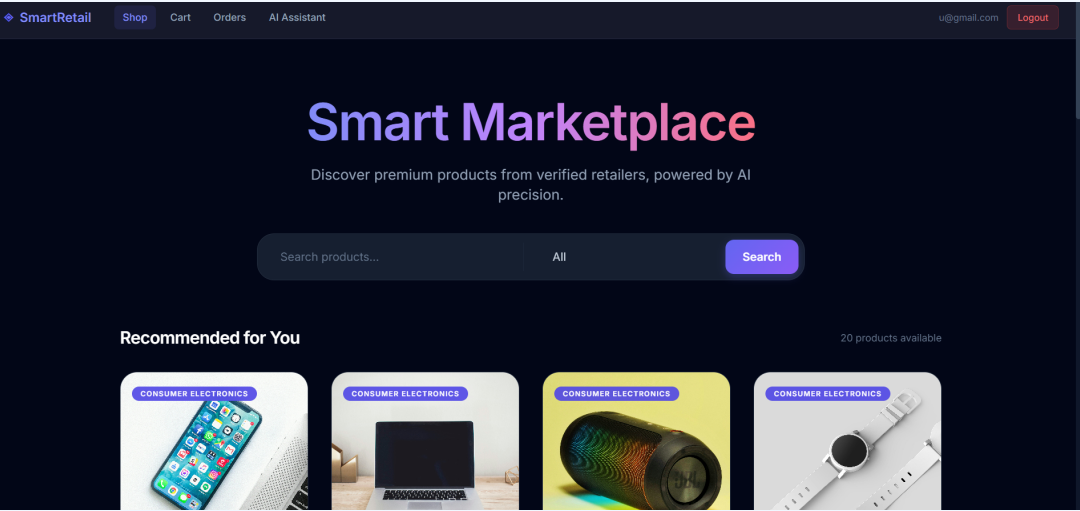
Registration Screen



New user registration with role selection (Customer, Retailer, or Admin).

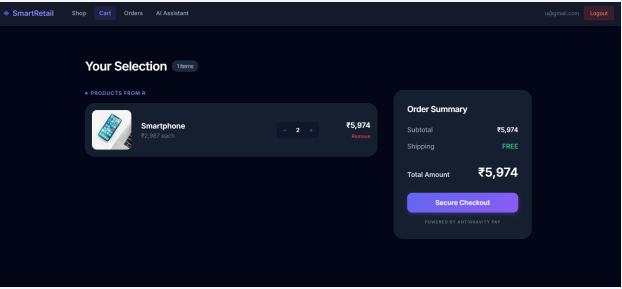
Customer Portal

Customer Dashboard



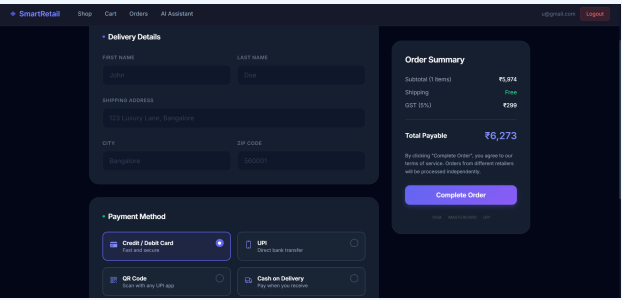
Overview of active orders, recent purchases, and quick-access links to catalog and cart.

Shopping Cart



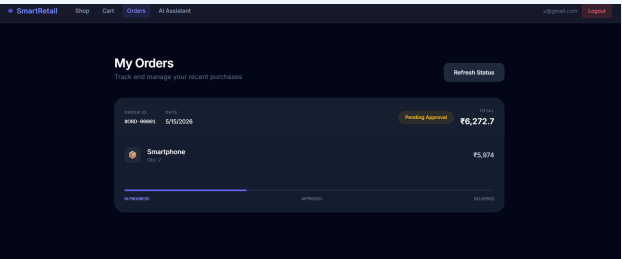
Add, update, and remove items. Live total calculation before proceeding to checkout.

Payments



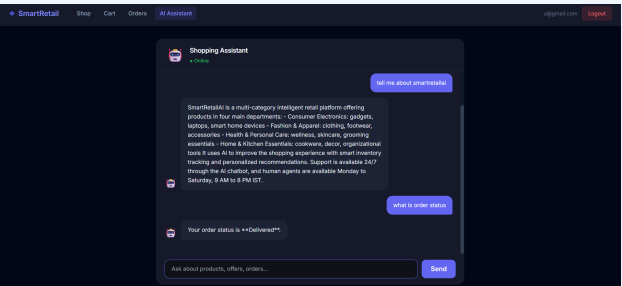
Order-level payment records with amount, method, and status tracking.

Order History



Full order history with status indicators (pending, processing, shipped, delivered).

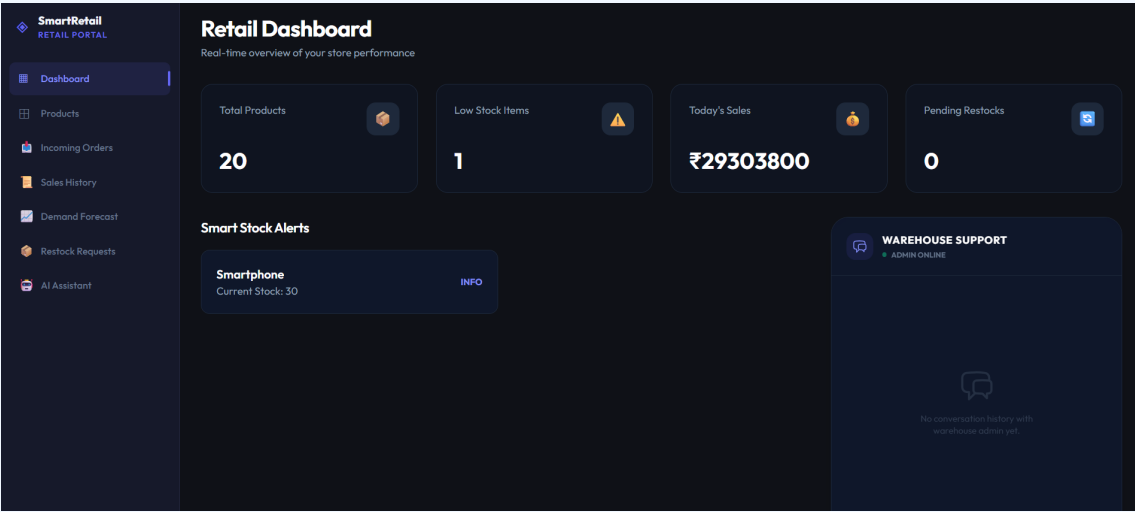
Customer AI Assistant



QA chatbot powered by LangGraph + RAG; answers policy, product, cart, and shipping questions.

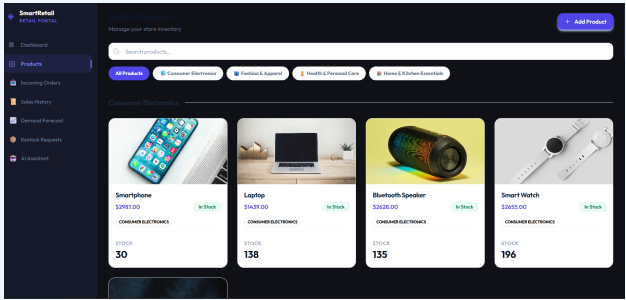
Retail Portal

Retailer Dashboard



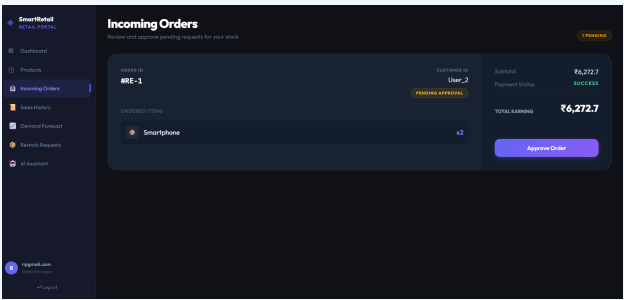
At-a-glance KPIs: total revenue, active products, pending restocks, and recent order counts.

Product Catalogue Management



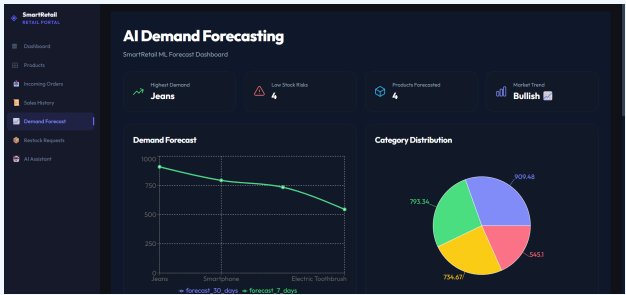
Add, edit, and remove retailer products exposed to customers. Includes category, price, and stock fields.

Incoming Orders



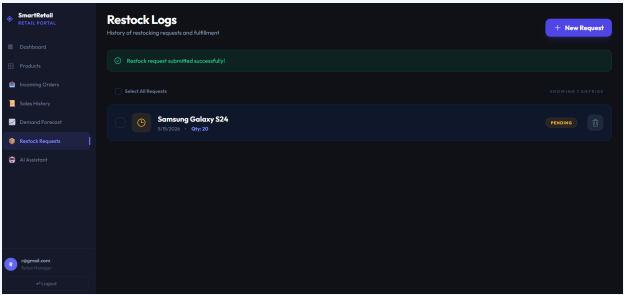
Retailer view of all customer orders with status management and fulfilment controls.

Demand Forecasting



7-day and 30-day Prophet forecasts with trend direction and stock health indicators per product.

Restock Requests



Submit and track inventory replenishment requests to the warehouse admin for approval.

Sales History

ORDER ID	DATE	CUSTOMER	ITEMS	AMOUNT	STATUS
ABC-1	12/15/2024	User_2	Smartphone	€1,232.7	Delivered

Historical sales data with revenue, quantity, and product-level breakdown for analytics.

Retail AI Assistant

SmartRetail AI Assistant
Capabilities - Analytics & AI

Forecast | Product Trends | Detect Anomalies | Restock Suggestions | Inventory Analysis

Hello! I'm your SmartRetail AI Assistant 🤖

I can help with:

- Demand Forecasting
- Product Trend Analysis
- Anomaly Detection
- Inventory Insights
- Restock Recommendations

Predict Smartphone demand next month

Smartphone demand next month is forecast at ***128.52 units***.

Trend High demand

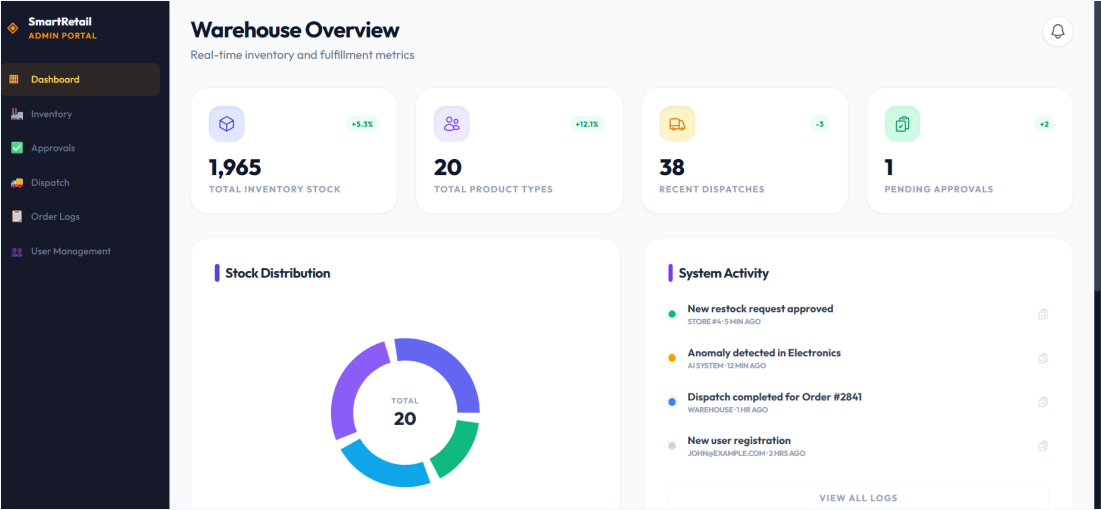
Action Restock soon*** to avoid stockouts.

Ask about forecasting, trends, anomalies...

Analytics + ML agent: natural-language queries about stock, revenue, forecasts, and anomalies.

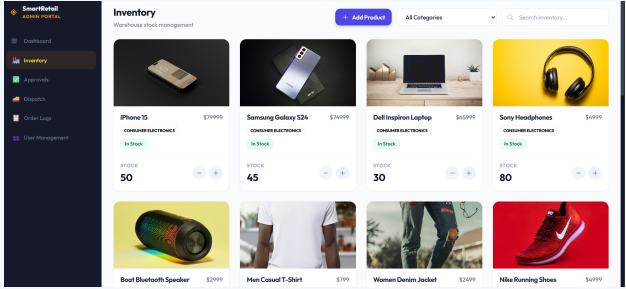
Warehouse Admin Portal

Warehouse Dashboard



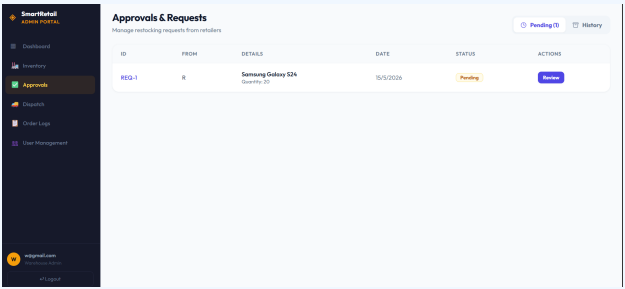
Central admin view: warehouse stock levels, pending approvals, dispatch queue, and user activity.

Inventory Management



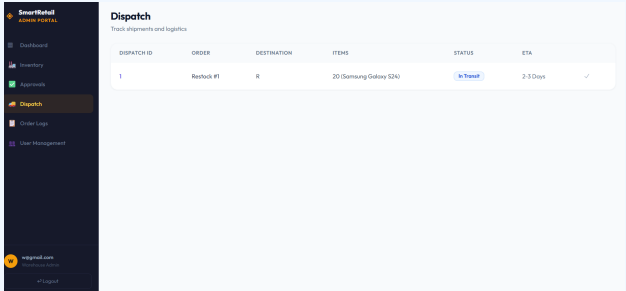
Full warehouse inventory CRUD: add, edit, and retire products with quantity and category controls.

Restock Approvals



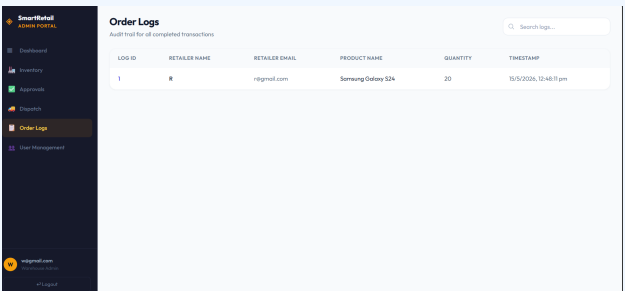
Review and approve or reject retailer restock requests with status and notes.

Dispatch Management



Track outgoing shipments from warehouse to retailer. Update dispatch status in real time.

Order Logs



Audit trail of all order events across the platform for compliance and investigation.

User Management

SmartRetail

ADMIN PORTAL

Dashboard

Inventory

Approvals

Dispatch

Order Logs

User Management

w@gmail.com

Warehouse Admin

Logout

Retailer Management

Manage retailers buying warehouse stock

RETAILER	EMAIL	ROLE	STATUS	JOINED	ACTIONS
R Retailer	r@gmail.com	retail	Active	May 15, 2026	

Admin panel to view, activate, deactivate, and manage all platform users across all roles.

13. Data Model & Business Workflows

Entity	Purpose
User	Stores email, password hash, role, status, and creation timestamp for customer, retail, and admin identities.
WarehouseProduct	Master warehouse inventory with name, price, quantity, image, and category.
RetailerProduct	Products owned by a retailer and exposed to customers for purchase.
CartItem	Customer cart entries tied to retailer products.
Order / OrderItem	Completed checkout records with retailer, customer, item quantity, and purchase price.
Payment	Order-level payment record with amount, status, method, and timestamp.
RestockRequest	Retailer-to-admin inventory replenishment request with status and notes.
Message	Retailer/admin communication messages with read status.
Sale	ML-ready sales records including revenue, profit, stock, discounts, rolling averages, supplier, warehouse, and region.
ForecastPrediction / AnomalyAlert	Stores prediction and anomaly alert outputs for forecasting workflows.

14. Challenges & Optimizations

Challenge	Solution
Multi-role Complexity	Separated customer, retail, and admin pages into dedicated portal folders and enforced roles through ProtectedRoute.
AI Routing Reliability	Used explicit role and keyword routing in the orchestrator to route predictable retail queries to the right agent.
RAG Availability	Implemented Pinecone retrieval with local document fallback so policy answers still work when cloud vector search is unavailable.
Tool-Grounded Analytics	Analytics answers are backed by SQLAlchemy queries instead of generic LLM guesses.
Forecast Model Access	Mapped friendly product names to model IDs and wrapped model calls in LangChain tools for the retail ML agent.
Deployment Simplicity	Bundled React build output into the FastAPI container, allowing one deployable artifact.
Database Portability	DATABASE_URL configuration and pyodbc support allow development on SQLite and migration toward Azure SQL.

15. Future Enhancements

- Add streamed chatbot responses with Server-Sent Events or WebSockets for a smoother AI experience.
- Persist conversation history per user so agents can maintain context across sessions.
- Expand forecasting coverage beyond current featured products and store model metadata in the database.
- Move forecast and anomaly model paths into environment configuration for cleaner container deployment.
- Add automated tests for route permissions, checkout behaviour, restock transitions, and AI routing decisions.
- Introduce background jobs for scheduled forecast refreshes and anomaly scans.
- Add richer admin audit logs for inventory changes, approvals, dispatches, and user management.
- Use Azure AI Search or a managed Pinecone deployment strategy for scalable production RAG.
- Add role-specific notification delivery for restock approvals, order status changes, and low-stock alerts.

16. Conclusion

Smart Retail AI successfully demonstrates how modern AI engineering can be embedded into a practical retail management platform. The project combines transactional commerce workflows, role-based dashboards, protected APIs, machine-learning forecasts, anomaly detection, RAG-powered support, LangGraph orchestration, and cloud-ready deployment into a cohesive full-stack application.

The platform goes beyond a traditional retail CRUD system by giving customers AI-assisted support, giving retailers business analytics and forecasting intelligence, and giving administrators operational control over warehouse inventory, approvals, dispatch, logs, and user management. Its architecture is modular enough to grow into a production retail assistant with deeper forecasting, richer analytics, and stronger cloud automation.

Prepared for: Left Shift Program 2026 – Data & AI (T5)	Domain: Intelligent Smart Retail System	Platform: Smart Retail AI
---	--	---------------------------